

# Ingeniería Directa

## Contenido

|                                                        |    |
|--------------------------------------------------------|----|
| Ingeniería Directa .....                               | 2  |
| Diagrama de clases .....                               | 2  |
| Clase .....                                            | 2  |
| Relaciones .....                                       | 4  |
| • Asociación.....                                      | 4  |
| • Agregación.....                                      | 8  |
| • Composición .....                                    | 8  |
| • Dependencia .....                                    | 9  |
| • Generalización .....                                 | 11 |
| • Realización .....                                    | 14 |
| • Comparación entre generalización y realización ..... | 16 |
| Clases Abstractas.....                                 | 16 |
| Diagrama de Secuencia .....                            | 20 |
| Ejemplo: Facturar.....                                 | 22 |
| Diagrama de Clase .....                                | 22 |
| Código Java.....                                       | 23 |
| Bibliografía .....                                     | 29 |
| Contribuciones .....                                   | 29 |

## Ingeniería Directa

Debido a que UML es un lenguaje de modelado, existe un mapeo significativo cuando se convierten los modelos en UML a un lenguaje de programación.

Al proceso de comenzar a generar código a través de modelos se le conoce como ingeniería directa.

El propósito de este documento es establecer correspondencias entre los diferentes diagramas UML y el lenguaje de programación Java

## Diagrama de clases

Un diagrama que muestra un conjunto de clases, interfaces y sus contenidos, y las relaciones entre éstos; los diagramas de clases muestran el diseño de un sistema desde un punto de vista estático.

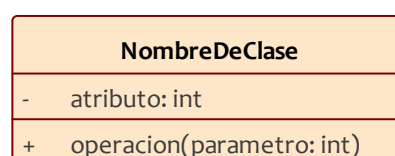
Los diagramas de clases tienen los siguientes elementos:

- Clases: atributos, operaciones y visibilidad.
- Interfaces
- Relaciones: Herencia, Realización, Composición, Agregación, Asociación y Uso.
- Responsabilidades: son las obligaciones de una clase. No son lo mismo que los métodos. Estos se implementan para llevar a cabo responsabilidades.

## Clase

Una clase es una descripción de un conjunto de objetos que comparten los mismos atributos, operaciones, relaciones y semántica. Representa un concepto discreto dentro de la aplicación que se está modelando: una cosa física (como un aeroplano), una cosa de negocios (como un pedido), una cosa lógica (como un horario de difusión), una cosa de una aplicación (como un botón de cancelar), una cosa del computador (como una tabla hash), o una cosa del comportamiento (como una tarea). Las clases son los focos alrededor de los cuales se organizan los sistemas orientados a objetos.

En UML, una clase se representa como un rectángulo con tres compartimientos, en el primer compartimento va el nombre de la clase; el segundo compartimento corresponde a los atributos de la clase, los cuales equivalen a variables de instancia en Java; y en el tercero, las operaciones de la clase, estas en Java se mapean a métodos de clase.



Cada clase UML se corresponde con un fichero de extensión .java que contiene el código fuente de la definición de propiedades y métodos de clase. La estructura sencilla sería la siguiente:

```
public class NombreDeClase {  
  
    //Atributos  
    private1 int2 atributo;  
    // Constructor de la clase  
    public NombreDeClase () {  
        .....  
    }  
    //Operaciones  
    public1 void3 operación(4)  
    {  
        .....  
    }  
}
```

**1- Visibilidad. Puede ser:**

- + : *public*
- # : *protected*
- : *private*

**2- Tipo de dato Java. Puede ser simple (char,int,etc..), perteneciente a una clase java (Vector, Hashtable, etc.) o definido por el programador.**

**3- Tipo de datos Java de retorno del método de clase. En caso de que el método no devuelva ningún tipo de datos se debe utilizar "void".**

**4- Tipo de datos y nombre de los parámetros del método de clase separados por comas.**

○ **Ejemplo de Clase Java:**

Ahora veamos un ejemplo de una clase "Producto":

| Producto                                                                                                                                                                                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| - codigo: int<br>- nombre: String<br>- precio: double                                                                                                                                         |
| + crear()<br>+ getCodigo(): int<br>+ setCodigo(newVal: int): void<br>+ getNombre(): String<br>+ setNombre(newVal: String): void<br>+ getPrecio(): double<br>+ setPrecio(newVal: double): void |

```
public class Producto {  
  
    private int codigo;  
    private String nombre;  
    private double precio;  
  
    public Producto () { };  
  
    public void setCodigo ( int newVar ) {  
        codigo = newVar;  
    }  
  
    public int getCodigo ( ) { return  
        codigo;  
    }  
  
    public void setNombre ( String newVar ) {  
        nombre = newVar;  
    }  
  
    public String getNombre ( ) {  
        return nombre;  
    }  
  
    public void setPrecio ( double newVar ) {  
        precio = newVar;  
    }  
  
    public double getPrecio ( ) {  
        return precio;  
    }  
}
```

## Relaciones

Una relación es una conexión entre elementos. En las relaciones se habla de una clase destino y de una clase origen. La clase origen es desde la que se realiza la acción de relacionar, es decir, desde la que parte la flecha. La clase destino es la que recibe la flecha. Las relaciones se pueden modificar con estereotipos o con restricciones.

Comprender el significado de las relaciones en un modelo es el primer paso para entender cómo se implementarán en código Java.

Los tipos de relaciones son los siguientes:

- **Asociación**

Especifica que los objetos de una clase están relacionados con los objetos de otra clase. **Esta relación perdura en el tiempo.**

Se representa mediante una línea continua, que une las dos clases. Podemos indicar

el nombre, multiplicidad en los extremos, su rol o papel que desempeña cada una de ellas y sentido de la navegación.

En UML se representa del siguiente modo:



Veamos detalladamente la especificación de la asociación.

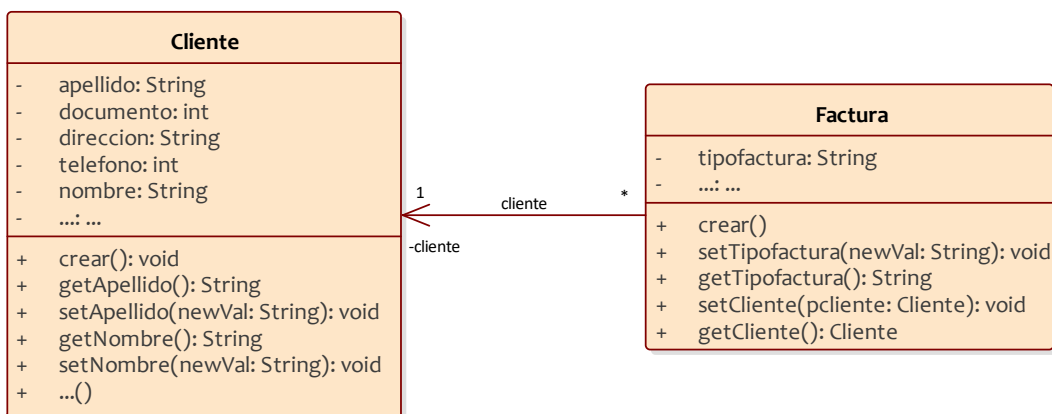
- Nombre: Nombre de la asociación.
- Clases asociadas: Clases que intervienen en la asociación
- Rol: Cada uno de los extremos de la asociación.
- Multiplicidad: Definen cuantas instancias de la clase 1 se relacionan con la clase 2.
- Navegabilidad: Es una propiedad del rol que indica que es posible navegar unidireccionalmente a través de la asociación desde la clase origen a la clase destino.

Según la navegabilidad de la asociación la podemos clasificar en dos grupos:

## Unidireccional

Es la relación que especifica la navegación en un solo sentido entre las clases conectadas. La clase hacia donde se dirige la navegación de la relación se convierte en una variable de instancia de la clase que da origen a la relación.

### Unidireccional con cardinalidad a 1



```
public class Cliente {
    private String apellido;
    private int documento;
    private String direccion;
    private int telefono;
    private String nombre;
```

```
/* Otros atributos
.....
*/

public Cliente () { };

public void setApellido ( String newVar ) {
    apellido = newVar;
}

public String getApellido ( ) {
    return apellido;
}
/* Otros métodos
...
*/
}

public class Factura {

    private Cliente cliente;
    private String tipofactura;

    /* Otros atributos
    .....
    */

    public Factura () { };

    public void setCliente( Cliente pcliente )
    {
        cliente = pcliente;
    }

    public Cliente getCliente()
    {
        return cliente;
    }

    /* Otros métodos
    .....
    */
}
```

Los métodos “setCliente” y “getClientes” se utilizan para asignar o hacer referencia al objeto Cliente relacionado con la factura.

## Unidireccional con cardinalidad a n

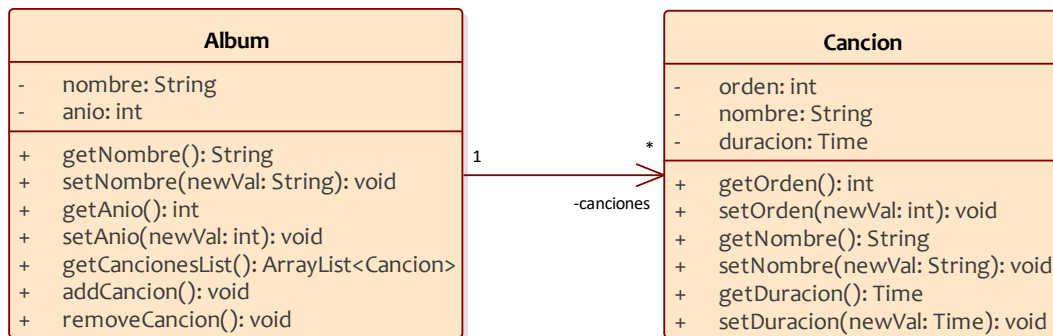
Para representar la cardinalidad con grado mayor a 1 en el código Java, se puede usar un atributo de tipo “lista” (array) que contenga los objetos de la clase correspondiente:

```
private List<Cancion> canciones = new ArrayList<Cancion>();
```

No es necesario que un conjunto de elementos se defina siempre de este modo, también

existen otros tipos de datos (colecciones de Java) que permiten trabajar con grupos de objetos y representar cardinalidad indefinida “n”, por ejemplo:

```
private Vector<E> rol;           //Donde E representa el tipo
private ArrayList<E> rol;       //de elementos de la colección
```



```
public class Cancion {

    private int orden;
    private String nombre;
    private Time duracion;
    /* Otros atributos
    .....
    */

    public Cancion () { };

    public void setNombre ( String pnombre ) {
        nombre = pnombre;
    }

    public String getNombre ( ) {
        return nombre;
    }
    /* Otros métodos
    ...
    */
}

public class Album {

    private String nombre;
    private int anio;
    private List<Cancion> canciones = new ArrayList<Cancion>();

    /* Otros atributos
    .....
    */

    public Album () { };

    public void addCancion( Cancion cancion )
    {

```

```
    canciones.add(cancion) ;  
}  
  
public List getCancionesList()  
{  
    return canciones;  
}  
  
/* Otros métodos  
.....  
*/  
}
```

El método “addCancion” se utiliza para asignar un nuevo objeto Canción a la lista de canciones y “getCancionesList” se utiliza para hacer referencia a los objetos Canción relacionados con el Álbum.

Es importante que cuando definimos el método get de un rol con cardinalidad a n, el mismo sea definido como “getRolList” añadiendo al final de la signatura del mismo la palabra List. Esto permite denotar que ese rol constituye una lista o arreglo de objetos de la clase a la que apunta la asociación.

## Bidireccional

Es una asociación que implica la navegación en ambos sentidos entre las clases conectadas. Corresponde a dos asociaciones unidireccionales en sentidos opuestos. En general, **no son recomendables** y la mayoría de las asociaciones en los diagramas de clases deberían ser unidireccionales.

- **Agregación**

Es una forma de asociación que representa una relación “es parte de” entre dos clases, donde una clase es el “todo” y otra la “parte”. No tiene significado adicional en el lenguaje Java, por lo que al codificar es igual que una asociación unidireccional.

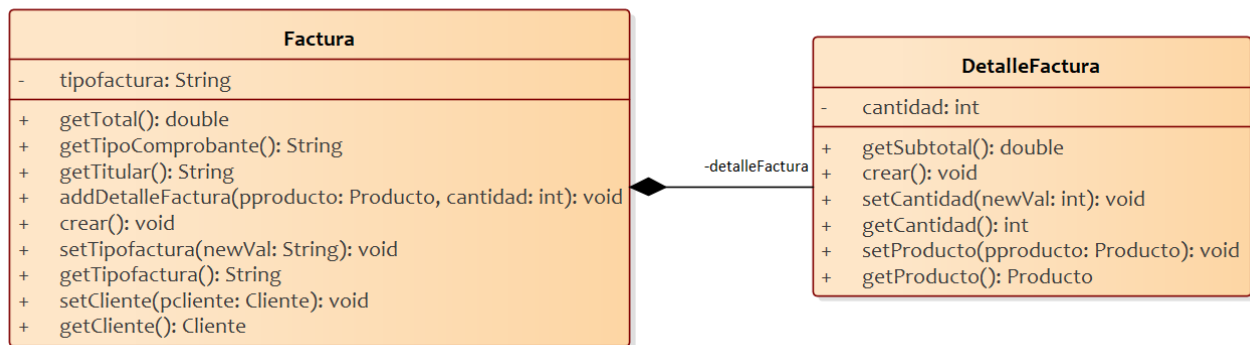
- **Composición**

Las composiciones son asociaciones que representan uniones muy fuertes. Dichas relaciones son tan fuertes que las partes no pueden existir por sí mismas. Únicamente existen como parte del conjunto, y si este es destruido las partes también lo son.

Implica que ninguna otra clase debería conocer como está compuesto el objeto: la composición crea un “envoltorio” en donde los cambios en la composición del objeto no afectar al resto de las clases.

Implementar una composición en Java es equivalente a una asociación unidireccional, con la diferencia de que la creación de la clase pieza debería realizarse dentro del código de la clase contenedora. Por ejemplo:





```

public class Factura {

    private List <DetalleFactura> detalleFactura = new ArrayList<DetalleFactura>();

    public Factura () { };
    ...
    public void addDetalleFactura( Producto pproducto, int cantidad ){

        DetalleFactura daux = new DetalleFactura();
        daux.setProducto(pproducto);
        daux.setCantidad(cantidad);

        detallefactura.add(daux);
    }
    ...
}
    
```

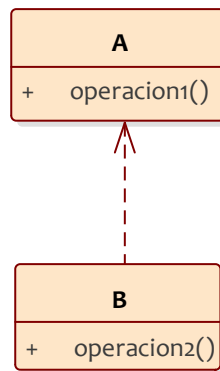
## • Dependencia

Es una relación que traduce algún tipo de referencia a una clase u objeto que no existe dentro del contexto.

No hay que confundirlo con las asociaciones, estos objetos no tienen una relación permanente, sino que son usados por ella ocasionalmente. Mientras que las asociaciones representan visibilidad de atributo (existe una variable de instancia del tipo de la clase a la que está asociada), la dependencia hace referencia a una visibilidad local.

Recordemos que una dependencia especifica una ruta por la cual un objeto puede enviar un mensaje a otro. Por tanto si hay una relación de dependencia entre una clase “B” y una clase “A” debe existir una referencia de A en el código de la clase B.

Suponiendo que operación es el método que hace referencia a la clase A.



```

public class A {
    public A () { };
    public void operacion1( ) {

    }
}

```

```

public class B {

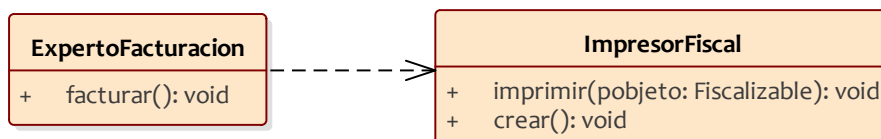
    public B () { };

    public void operacion2( ) {

        A a = new A();
        a.operacion1();

    }
}

```



```

public class ExpertoFacturacion {

    public ExpertoFacturacion () {

    };

    public void facturar() {

        Cliente cliente= new Cliente();
        Producto producto1 = new Producto();
        String fnumero;
        String ffecha; String
        ftfactura;
        .....
        ImpresorFiscal impresor = new ImpresorFiscal();
        impresor.imprimir(factura);

    }
}

```

```
public class ImpresorFiscal {  
  
    public ImpresorFiscal () { };  
  
    public void imprimir( Fiscalizable pobjeto )  
    {  
        /*Codigo Imprimir */  
    }  
  
    public void ImpresorFiscal( ) {  
    }  
}
```

## • Generalización

La relación de generalización es una relación taxonómica entre una descripción más general (clase padre) y una descripción más específica que se construye sobre ella y va extendiendo (clase hija). La descripción más específica es completamente consistente con la más general (tiene todas sus propiedades, miembros y relaciones) y puede contener información adicional.

Una clase hija es una clase más especializada que la clase padre.

**Propósito de la generalización.** La generalización tiene dos propósitos:

- El primero es el principio de sustitución. La regla es que una instancia de un descendiente puede ser utilizada dondequiera siempre que se use el antecesor
- La generalización permite operaciones polimórficas es decir, operaciones cuya implementación (método) se determina por la clase de objeto a la que se aplican, en lugar de ser explícitamente indicadas por el apuntador.

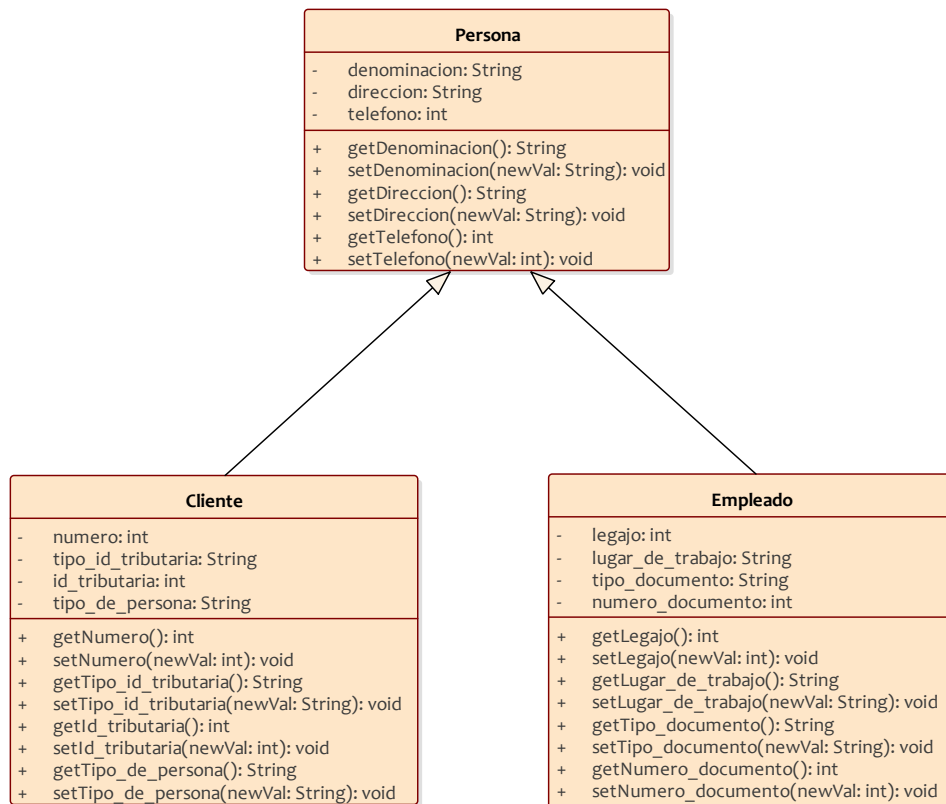
*Nota: en el diagrama de clases, los atributos, métodos y relaciones de una clase se muestran en el nivel más alto de la jerarquía en la que son aplicables. Si un método figura en la superclase y en la subclases, significa que el método esta sobrescrito.*

La forma de codificar esta relación en Java es acompañar a la cabecera de la definición de la subclase con la palabra “*extends*”, seguida del nombre de la superclase:

```
public class NombreSubClase extends NombreSuperclase
```

Una de las características importantes de Java es que no permite herencia múltiple, aunque UML si lo permite. Como solución a este problema se puede aplicar el concepto de interface de Java.

### ○ Ejemplo de Generalización:



```
public class Persona {

    private String denominacion;
    private String direccion;
    private int telefono;

    public Persona () { };

    public void setDenominacion ( String newVar ) {
        denominacion = newVar;
    }

    public String getDenominacion ( ) {
        return denominacion;
    }

    public void setDireccion ( String newVar ) {
        direccion = newVar;
    }

    public String getDireccion ( ) {
        return direccion;
    }

    public void setTelefono ( int newVar ) {
        telefono = newVar;
    }

    public int getTelefono ( ) {
        return telefono;
    }
}
```

```
}

public class Cliente extends Persona {

    private int numero;
    private String tipo_id_tributaria;
    private int id_tributaria;
    private String Tipo_de_Persona;

    public Cliente () { };

    public void setNumero ( int newVar ) {
        numero = newVar;
    }

    public int getNumero ( ) { return
        numero;
    }

    public void setTipo_id_tributaria ( String newVar ) {
        tipo_id_tributaria = newVar;
    }

    public String getTipo_id_tributaria ( ) {
        return tipo_id_tributaria;
    }

    public void setId_tributaria ( int newVar ) {
        id_tributaria = newVar;
    }

    public int getId_tributaria ( ) {
        return id_tributaria;
    }

    public void setTipo_de_Persona ( String newVar ) {
        Tipo_de_Persona = newVar;
    }

    public String getTipo_de_Persona ( ) {
        return Tipo_de_Persona;
    }
}

public class Empleado extends Persona {

    private int legajo;
    private String lugar_de_trabajo;
    private String tipo_documento;
    private int numero_documento;

    public Empleado () { };

    public void setLegajo ( int newVar ) {
        legajo = newVar;
    }

    public int getLegajo ( ) { return
```

```
        legajo;
    }

    public void setLugar_de_trabajo ( String newVar ) {
        lugar_de_trabajo = newVar;
    }

    public String getLugar_de_trabajo ( ) {
        return lugar_de_trabajo;
    }

    public void setTipo_documento ( String newVar ) {
        tipo_documento = newVar;
    }

    public String getTipo_documento ( ) {
        return tipo_documento;
    }

    public void setNumero_documento ( int newVar ) {
        numero_documento = newVar;
    }

    public int getNumero_documento ( ) {
        return numero_documento;
    }
}
```

- **Realización**

Es la relación entre 2 elementos en donde uno de los elementos representa una interfaz que otro elemento (clase) debe implementar.

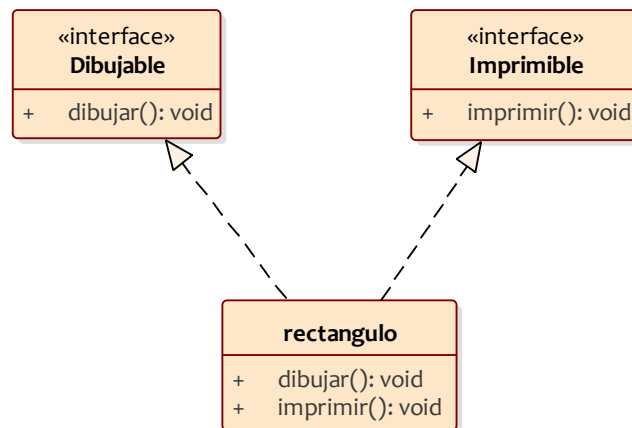
Una interfaz es una colección de operaciones que son utilizadas para especificar un servicio de una clase. Estas definen un contrato, en el sentido que una clase que “realiza” o “implementa” una interfaz, debe proporcionar los métodos que están definidos en la misma. Las interfaces constituyen una forma de separar la especificación de la funcionalidad de sus implementaciones.

En UML, la realización se representa por una línea punteada con una flecha vacía apuntando al objeto abstracto.

Para crear una interfaz en Java, se utiliza la palabra clave “*interface*” en lugar de “*class*”. La interfaz puede definirse “*public*” o sin modificador de acceso, y tiene el mismo significado que para las clases. Todos los métodos que declara una interfaz son siempre “*public*”.

Una interfaz puede también contener datos miembro, pero estos son siempre “*static*” y “*final*”.

Para indicar que una clase implementa los métodos de una interfaz se utiliza la palabra clave “*implements*” seguido de una lista separada por comas de los nombres de las interfaces que implementa.

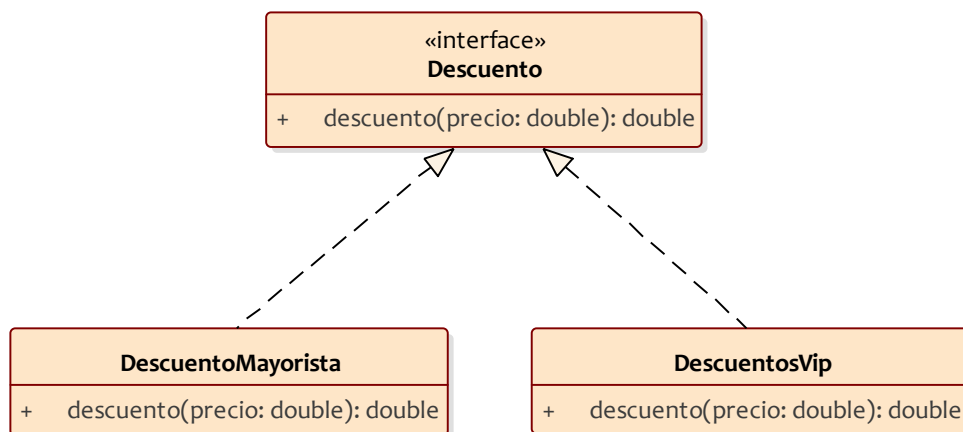


```
public interface Dibujable {
    public void dibujar( );
}

public interface Imprimible {
    public void imprimir( );
}

public class rectangulo implements Imprimible, Dibujable {
    public rectangulo () { };

    public void imprimir( )
    {
        .....
    }
    public void dibujar( )
    {
        .....
    }
}
```



```
public interface Descuento {  
  
    public double descuento( double precio )  
  
}  
  
public class DescuentoMayorista implements Descuento {  
  
    public DescuentoMayorista () { };  
  
    public double descuento( double precio )  
    {  
        return precio * 0.5  
    }  
  
}  
  
public class DescuentosVip implements Descuento {  
  
    public DescuentosVip () { };  
  
    public double descuento( double precio ){  
  
        return  precio * 0.15  
    }  
  
}
```

- **Comparación entre generalización y realización**

Tanto la generalización como la realización relacionan una descripción más general, con versiones más detalladas de la misma.

La generalización relaciona dos elementos en el mismo nivel semántico (dos clases, dos interfaces). Esta es una relación jerárquica entre una descripción más general y otra más específica que se construye sobre ella y la extiende. La descripción más específica es completamente consistente con la más general (tiene todas sus propiedades, miembros y relaciones), y puede contener información adicional.

Por el otro lado, la realización conecta dos elementos en diversos niveles semánticos, como una interfaz con una clase, en donde uno especifica el comportamiento del otro pero no su estructura e implementación.

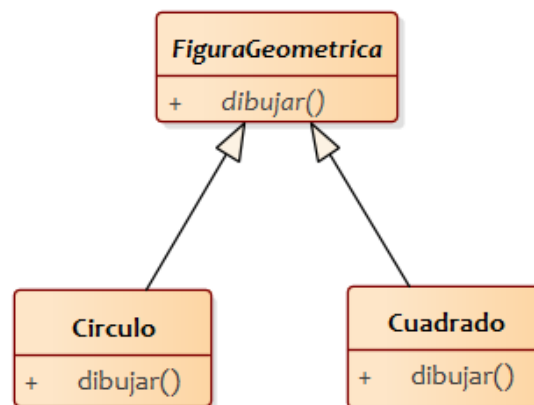
## **Clases Abstractas**

Una clase abstracta es una clase no instanciable —es decir, que no puede tener instancias directas, bien porque su descripción está incompleta (como cuando faltan métodos para una o más operaciones), o bien porque no se tiene la intención de instanciarla aunque su descripción esté completa. Una clase abstracta está pensada para la especialización. Para que sea útil, una clase abstracta debe tener descendientes que puedan tener instancias; una clase abstracta hoja carece de utilidad. (Puede aparecer como hoja en un marco de trabajo, pero finalmente deberá ser especializada.)

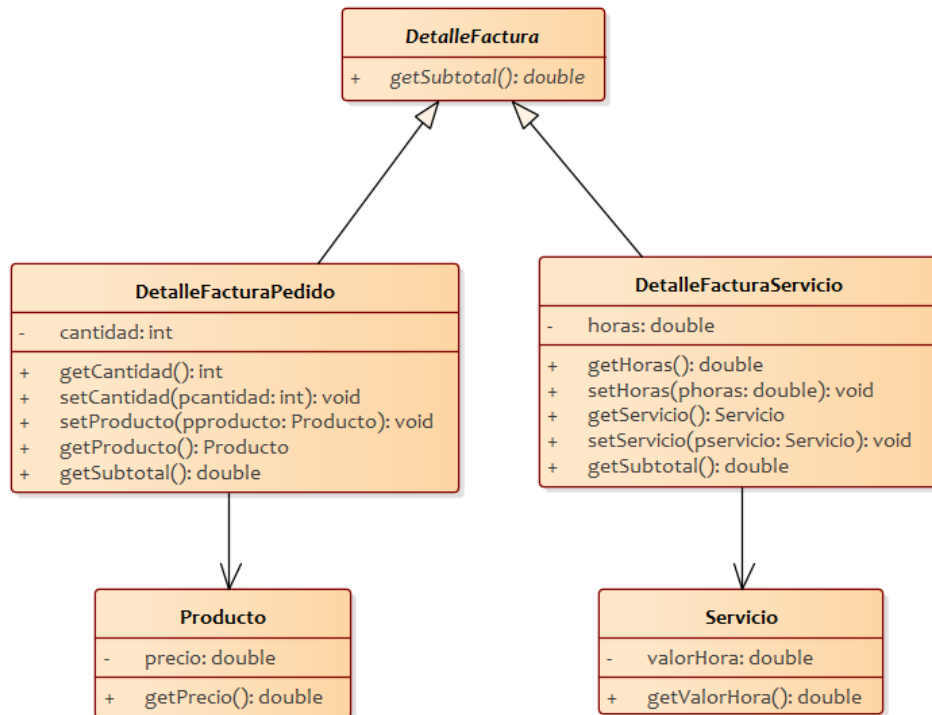


En UML, se puede especificar que una clase es abstracta escribiendo su nombre en cursiva. Esta convención se aplica a las operaciones y significa que dicha operación proporciona una signatura, pero por lo demás está incompleta y por tanto debe ser implementada por algún método a un nivel más bajo de abstracción.

La clase abstracta se declara simplemente con el modificador “*abstract*” en su declaración. Los métodos abstractos se declaran también con el mismo modificador, declarando el método pero sin implementarlo (sin el bloque de código encerrado entre {}). La clase derivada se declara e implementa de forma normal, como cualquier otra. Sin embargo, si no declara e implementa los métodos abstractos de la clase base (en el ejemplo el método dibujar), el compilador genera un error indicando que no se han implementado todos los métodos abstractos y que, o bien, se implementan, o bien se declara la clase abstracta.



```
public abstract class FiguraGeometrica {
    . . .
    abstract void dibujar();
    . . .
}
public class Circulo extends FiguraGeometrica {
    . . .
    void dibujar() {
        // codigo para dibujar Circulo
        . . .
    }
}
public class Cuadrado extends FiguraGeometrica {
    . . .
    void dibujar() {
        // codigo para dibujar Cuadrado
        . . .
    }
}
```



```

public abstract class DetalleFactura {

    public DetalleFactura () { };

    abstract public double getSubtotal( );

}

public class DetalleFacturaProducto extends DetalleFactura {

    private int cantidad;
    private Producto producto

    public DetalleFacturaProducto () { };

    public void setCantidad ( int pcantidad ) {
        cantidad = pcantidad;
    }

    public int getCantidad ( ) {
        return cantidad;
    }

    public void setProducto(Producto pproducto){
        producto=pproducto;
    }

    public Producto getProducto() {
        return producto;
    }

}
    
```

```
    public double getSubtotal( )
    {
        return producto.getPrecio() * cantidad
    }
}

public class DetalleFacturaServicio extends DetalleFactura {

    private double horas;
    private Servicio servicio;

    public DetalleFacturaServicio () { };

    public void setHoras ( double phoras ) {
        horas = phoras;
    }

    public void setServicio(Servicio pservicio){
        servicio=pservicio;
    }

    public Servicio getServicio(){
        return servicio;
    }

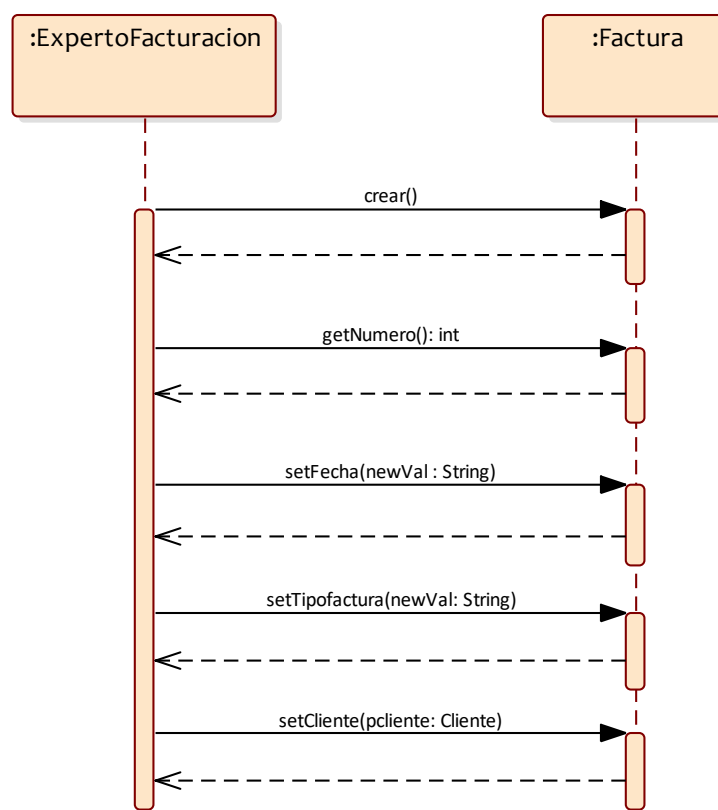
    public double getHoras ( ) {
        return horas;
    }

    public double getSubtotal( )
    {
        return servicio.getValorHora * horas
    }
}
```

## Diagrama de Secuencia

Los diagramas de secuencia son diagramas de interacción. Estos muestran elecciones en la asignación de responsabilidades a los objetos: se crean una vez que se han tomado las decisiones acerca de la asignación de responsabilidades, que se reflejan en los mensajes que se envían a diferentes clases de objetos.

Los diagramas de secuencia muestran las interacciones entre objetos organizadas en una secuencia temporal. En particular, muestra los objetos participantes en la interacción y la secuencia de mensajes intercambiados.



### ○ Ejemplo de ingeniería directa a partir de un Diagrama de secuencia

```
public class ExpertoFacturacion {  
  
    public ExpertoFacturacion () {  
  
    };  
    public void facturar(){  
  
        .....  
        .....  
        Factura factura = new Factura();  
        factura.getNumero();  
    }  
}
```

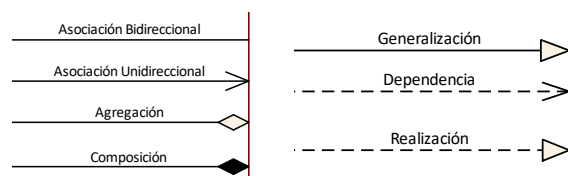
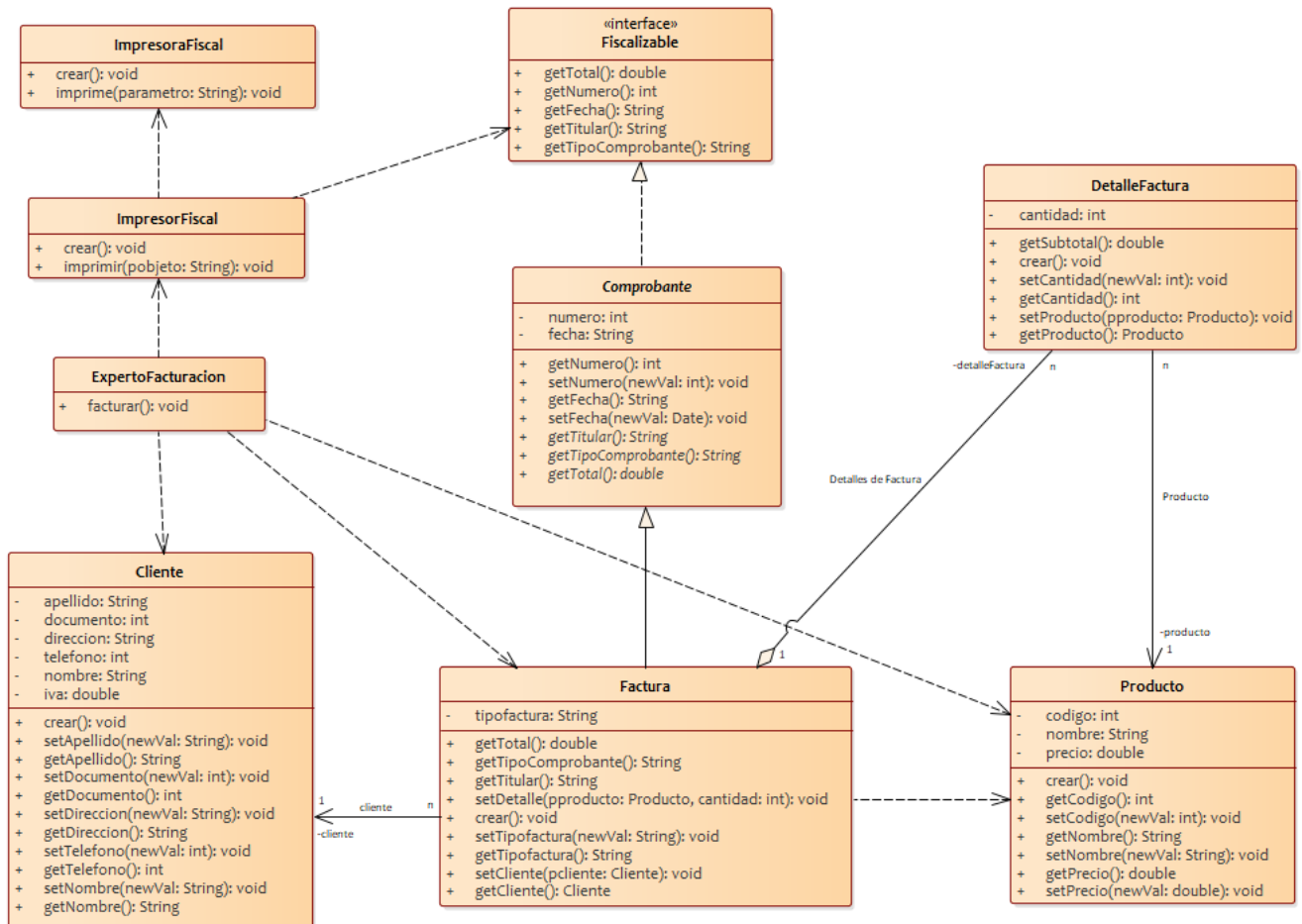
```
        factura.setFecha(ffecha);  
        factura.setTipofactura(ftfactura);  
        factura.setCliente(cliente);  
        .....  
    }  
}
```

## Ejemplo: Facturar

El siguiente ejemplo trata de la implementación de una versión simplificada de facturación.

Adjunto: Diagrama de secuencia

## Diagrama de Clase



## Código Java

### Clase ExpertoFacturacion

```
package ejemplo;

public class ExpertoFacturacion {

    public ExpertoFacturacion ()
    {

    };

    public void facturar(){

        Cliente cliente= new Cliente();
        Producto producto1 = new Producto();
        String fnumero;
        String ffecha; String
        ftfactura;
        .....
        .....
        .....
        .....
        Factura factura = new Factura();
        factura.setNumero(fnumero);
        factura.setFecha(ffecha);
        factura.setTipofactura(ftfactura);
        factura.setCliente(cliente);
        factura.setDetalle(producto1,cantidad1);
        ImpresorFiscal impresor = new ImpresorFiscal();
        impresor.imprimir(factura);
    }
}
```

### Clase Cliente

```
package ejemplo; public

class Cliente {

    private String apellido;
    private int documento;
    private String direccion;
    private int telefono;
    private String nombre;

    public Cliente () { };

    public void setApellido ( String newVar ) {
        apellido = newVar;
    }

    public String getApellido ( ) {
        return apellido;
    }
}
```

```
public void setDocumento ( int newVar ) {
    documento = newVar;
}

public int getDocumento ( ) {
    return documento;
}

public void setDireccion ( String newVar ) {
    direccion = newVar;
}

public String getDireccion ( ) {
    return direccion;
}

public void setTelefono ( int newVar ) {
    telefono = newVar;
}

public int getTelefono ( ) {
    return telefono;
}

public void setNombre ( String newVar ) {
    nombre = newVar;
}

public String getNombre ( ) {
    return nombre;
}
}
```

## Clase Producto

```
package ejemplo;

import java.util.*;

public class Producto {

    private int codigo;
    private String nombre;
    private double precio;

    public Producto () { };

    public void setCodigo ( int newVar ) {
        codigo = newVar;
    }

    public int getCodigo ( ) {
        return codigo;
    }

    public void setNombre ( String newVar ) {
        nombre = newVar;
    }
}
```



```
    }

    public String getNombre ( ) {
        return nombre;
    }

    public void setPrecio ( double newVar ) {
        precio = newVar;
    }

    public double getPrecio ( ) {
        return precio;
    }
}
```

## Clase Fiscalizable

```
package ejemplo;

public interface Fiscalizable {

    public double getTotal();

    public int getNumero();

    public String getFecha();

    public String getTitular();

    public String getTipoComprobante();

}
```

## Clase Comprobante

```
package ejemplo;

public abstract class Comprobante implements Fiscalizable {

    private int numero;
    private String fecha;

    public Comprobante () { };

    public void setNumero ( int newVar ) {
        numero = newVar;
    }

    public int getNumero ( ) {
        return numero;
    }

    public void setFecha ( String newVar ) {
        fecha = newVar;
    }
}
```

```
public String getFecha ( ) {  
    return fecha;  
}  
  
abstract public double getTotal( );  
  
abstract public String getTipoComprobante( );  
  
abstract public String getTitular( );  
}
```

## Clase Factura

```
package ejemplo; import  
  
java.util.*;  
  
public class Factura extends Comprobante {  
  
    private Cliente cliente; private  
    String tipofactura;  
    private List<DetalleFactura> detallefactura = new ArrayList<DetalleFactura>();  
  
    public Factura ( ) { };  
  
    public void setTipofactura ( String newVar ) {  
        tipofactura = newVar;  
    }  
  
    public String getTipofactura ( ) {  
        return tipofactura;  
    }  
  
    public double getTotal( ){  
  
        double total=0.0;  
        for(int i=0; i<detallefactura.size(); i++){  
            total +=((DetalleFactura) detallefactura.elementAt(i)).getSubtotal();  
        }  
  
        return total;  
    }  
  
    public String getTipoComprobante( ){  
        return tipofactura;  
    }  
  
    public String getTitular( ) {  
  
        return  cliente.getNombre() + ' ' + cliente.getApellido();  
    }  
  
    public void setDetalle( Producto pproducto, int cantidad ){  
  
        DetalleFactura daux = new DetalleFactura();  
        daux.setProducto(pproducto);
```

```
        daux.setCantidad(cantidad);

        detallefactura.add(daux);
    }

    public void removeDetalle (DetalleFactura detalle){
        detallefactura.remove(detalle);
    }

    public List getDetallesFacturaList ( ) {
        return detallefactura;
    }

    public void setCliente( Cliente pcliente ){

        cliente = pcliente;
    }

    public Cliente getCliente(){
        return cliente;
    }
}
```

## Clase DetalleFactura

```
package ejemplo; import

java.util.*;

public class DetalleFactura {

    private Producto producto;
    private int cantidad;

    public DetalleFactura ( ) { };

    public void setCantidad ( int newVar ) {
        cantidad = newVar;
    }

    public int getCantidad ( ) {
        return cantidad;
    }

    public double getSubtotal( )
    {
        return producto.getPrecio() * cantidad;
    }

    public void setProducto( Producto pproducto )
    {
        producto=pproducto;
    }

    public Producto getProducto( )
    {
        return producto;
    }
}
```

```
}
```

## Clase ImpresorFiscal

```
package ejemplo;

public class ImpresorFiscal {

    public ImpresorFiscal () { };

    public void imprimir( Fiscalizable pobjeto ) {

        String parametro;
        Integer numero = new Integer(pobjeto.getNumero());
        parametro = numero.toString() + ';' + pobjeto.getFecha() + ';' +
            pobjeto.getTipoComprobante() + ';' + pobjeto.getTitular()
            + ';' + pobjeto.getTotal(); ImpresoraFiscal
        impresora = new ImpresoraFiscal();
        impresora.Imprime(parametro);
    }
}
```

## Clase ImpresoraFiscal

```
package ejemplo;

public class ImpresoraFiscal {

    public ImpresoraFiscal () { };

    public void Imprime( String parametro ) {

        //Envia los datos al dispositivo
        System.out.println("Imprimiendo....");
        System.out.println(parametro);
    }
}
```

## Bibliografía

- Título: **El Lenguaje Unificado de Modelado. Manual de Referencia**; Autores: **James Rumbaugh, Ivar Jacobson, Grady Booch**; Editorial: Addison Wesley Longman Inc.; Año: 1999  
Edición en Español por Pearson Educación S.A., Año: 2000.
- Título: **UML y Patrones, Segunda Edición**; Autores: **Craig Larman**; Editorial: Pearson Education, Inc.; Año: 2002  
Edición en Español por Pearson Educación S.A., Año: 2003.

## Contribuciones

Ay. 2da Daniela Bertranou 2018  
Ay. 2da Gabriela Celeste Flores 2019